Semana 2. ESP32, tipos de datos y pulsadores

1. Del AVR al ESP32

En el ESP32 también se controlan los pines escribiendo bits en registros, aunque las señales pueden asignarse a diferentes GPIO mediante la **GPIO matrix**. La función **pinMode()** simplifica su configuración.

Función	AVR	ESP32
Entrada o salida	DDRx	GPIO_ENABLE_REG
Poner una salida en HIGH	PORTx	GPIO_OUT_W1TS_REG
Poner una salida en LOW	PORTx	GPIO_OUT_W1TC_REG
Leer entrada	PINx	GPIO_IN_REG

Los registros **W1TS/W1TC** permiten activar o limpiar bits con una sola escritura, sin tener que leer y reescribir todo el registro.

2. Pines seguros y alimentación

Los GPIO trabajan a **3,3 V** y no deben recibir 5 V directamente. Los GPIO **34–39** son solo de entrada y carecen de pull-up interna; **6–11** se usan con la memoria Flash. Conviene tener cuidado con los de arranque **0, 2, 5, 12 y 15** y con **1 y 3** (puerto serie). Para prácticas son útiles 16, 17, 18, 19 y 21.

Ejercicio: con alimentación de 3,3 V, un LED rojo de caída 2 V y resistencia de 330 Ω: I = (3,3 − 2) / 330 = **0,00394 A ≈ 4 mA**. Un botón en GPIO35 con INPUT_PULLUP no funciona con pull-up interna: necesita resistencia externa.

3. Arranque y variables

Después del encendido se ejecuta **setup()** una vez y **loop()** se repite. Una variable local normal se inicializa de nuevo en cada entrada a la función; una **static** mantiene su valor; una global también conserva su valor durante la ejecución.

Ejercicio: si en cada vuelta de loop() se incrementan un contador global, uno local que comienza en 0 y otro static que comienza en 0, en la tercera vuelta sus valores serán **3, 1 y 3**, respectivamente.

4. Tipos de datos y errores frecuentes

El tipo **int** tiene 2 bytes en Arduino Uno y 4 en ESP32. Para tamaños exactos se usan **uint8_t** (0 a 255), **int8_t** (−128 a 127), **uint16_t** y **uint32_t**. Un valor sin signo que supera el máximo vuelve a cero.

```
uint8_t x = 200, y = 100;
uint8_t r = x + y;         // 300 mod 256 = 44
uint8_t a = 0xCC;
uint8_t inv = (uint8_t)~a; // 0x33
uint32_t mascara = 1UL << 31;
```

Errores para corregir: for (uint8_t i=0; i<256; i++) no termina porque i vuelve a 0; usar int. Para guardar **millis()** se recomienda uint32_t, no int. Si una variable dentro de loop() debe recordar su valor, puede declararse static.

Ejemplo de desbordamiento: si uint8_t c empieza en 253 y se incrementa cinco veces, los valores que toma son **254, 255, 0, 1 y 2**. Esto ocurre porque solo caben 256 valores distintos.

5. Tiempo sin bloquear con millis()

delay() mantiene detenido el avance de loop(); **millis()** permite realizar otras tareas mientras se espera. Se comprueba el intervalo con la resta **ahora − anterior ≥ período** usando variables de tiempo sin signo.

```
uint32_t tAnterior = 0;
bool led = false;
void loop() {
  uint32_t ahora = millis();
  if (ahora - tAnterior >= 500) {
    tAnterior = ahora;
    led = !led; digitalWrite(16, led);
  }
}
```

En este ejemplo, **GPIO16 debe configurarse como OUTPUT en setup()**. El LED cambia cada 500 ms sin bloquear el programa.

6. Pulsadores: pull-up, flanco y rebote

Un botón entre **GPIO21 y GND**, usando INPUT_PULLUP, se lee HIGH cuando está suelto y LOW cuando está presionado. El **rebote** genera cambios rápidos durante una pulsación; se espera unos **30 ms** de estabilidad para aceptarla. Para contar una pulsación se detecta el cambio de HIGH a LOW, no todas las vueltas en LOW.

Ejemplo: mantener el botón presionado durante 200 ms no debe aumentar el contador en cada vuelta de loop(). El programa espera una transición estable de HIGH a LOW y registra una sola pulsación.

Aplicación (AP2): el juego de luces lee el pulsador, cambia la velocidad, usa millis() para avanzar la luz y actualiza los LED. La idea es separar **entrada → lógica → salida** sin detener la animación con delay().